

Sarah Benavides

CS 362 Lab 4

2/17/2026

Disk and File Systems

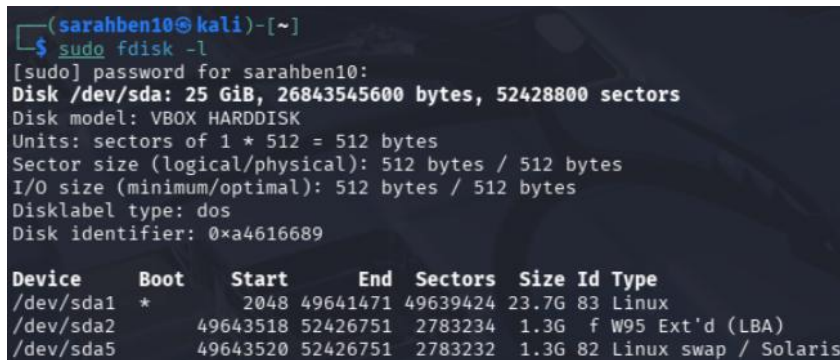
Part 1: Non-technical summary:

In this lab, we concentrated on the basics of how hard disks and file systems are examined during a digital forensics investigation. We began by identifying the list of hard drives, partitions, and USB devices within the Linux virtual machine. After comparing the information in the lab document to the information received in the first step of the lab, I found that the naming of the hard disks and drives is very different between Linux and Windows. We also looked at a few more different ways to display hard disk information using two other commands.

In another part of the lab, we experimented with creating hash values for strings, files, and entire directories. These hash values are similar to human fingerprints and are used to identify specific data and whether or not it has been altered. This lab also touched on disk image acquisition. This part allowed us to create copies of the components of a hard disk using specific commands in the Linux terminal, hashing the content in the file, and logging the process into a file. We wrapped this lab up by exploring the task of splitting large images into multiple separate smaller files and wiping the drives in a few different ways to prevent data recovery. Overall, this lab gave great hands-on experience in safely handling, confirming, and examining digital data.

Part 2: Technical summary:

To begin this lab, I opened VirtualBox and powered on my Kali Linux virtual machine. Once the virtual machine booted up, I opened a terminal and input the following command: **sudo fdisk -l**.



```
(sarahben10@kali)-[~]
$ sudo fdisk -l
[sudo] password for sarahben10:
Disk /dev/sda: 25 GiB, 26843545600 bytes, 52428800 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa4616689

Device     Boot      Start      End  Sectors  Size Id Type
/dev/sda1  *           2048 49641471 49639424 23.7G 83 Linux
/dev/sda2              49643518 52426751 2783234  1.3G  f W95 Ext'd (LBA)
/dev/sda5              49643520 52426751 2783232  1.3G  82 Linux swap / Solaris
```

In this process, we discover that the naming of the hard disks and drives is different from Windows. In Windows, drives are labeled as **Disk 0**, **Disk 1**, **Disk 2**, etc., but here in my Linux virtual machine, the drives are labeled as **/dev/sda1**, **/dev/sda2**, and **/dev/sda5**. Within this label, **sd** stands for SCSI Mass-Storage Driver, and the letter after **sd** represents the drive number. In our case, the drive numbers are **a1**, **a2**, and **a5**. In the screenshot above, the first drive is **sda**. **Sda1** is the first

partition on the sda drive, **sda2** is the second partition, and **sda5** is the third partition. Using the command **/dev**, the path of all the devices that the drive recognizes by Linux are listed.

```

└─$ cd /dev
└─(sarahben10@kali)-[/dev]
└─$ ls
autofs      mem         tty         tty32       tty57       vcs5
block       mqueue     tty0        tty33       tty58       vcs6
bsg         net        tty1        tty34       tty59       vcs7
btrfs-control null       tty10       tty35       tty6        vcsa
bus         nvram      tty11       tty36       tty60       vcsa1
cdrom       port       tty12       tty37       tty61       vcsa2
char        ppp        tty13       tty38       tty62       vcsa3
console     psaux      tty14       tty39       tty63       vcsa4
core        ptmx       tty15       tty4        tty7        vcsa5
cpu_dma_latency pts        tty16       tty40       tty8        vcsa6
cuse        random     tty17       tty41       tty9        vcsa7
disk        rfkill     tty18       tty42       ttyS0       vcsu
dma_heap    rtc        tty19       tty43       ttyS1       vcsu1
dri         rtc0       tty2        tty44       ttyS2       vcsu2
fb0         sda        tty20       tty45       ttyS3       vcsu3
fd          sda1       tty21       tty46       udmabuf     vcsu4
full        sda2       tty22       tty47       uhid        vcsu5
fuse        sda5       tty23       tty48       uinput      vcsu6
hidraw0     sg0        tty24       tty49       urandom     vcsu7
hpet        sg1        tty25       tty5        userfaultfd vfio
hugepages   shm        tty26       tty50       vboxguest   vga_arbiter
hwrng       snapshot   tty27       tty51       vboxuser    vhci
input       snd        tty28       tty52       vcs         vhost-net
kmsg        sr0        tty29       tty53       vcs1        vhost-vsock
log          stderr     tty3        tty54       vcs2        vmci
loop-control stdin      tty30       tty55       vcs3        vsock
mapper      stdout     tty31       tty56       vcs4        zero

```

The next command we experimented with is **sudo lshw -class disk -short**. This command displays the hard disk's information.

```

└─(sarahben10@kali)-[/dev]
└─$ sudo lshw -class disk -short
H/W path          Device          Class          Description
-----
/0/100/1.1/0.0.0  /dev/cdrom      disk           CD-ROM
/0/100/d/0.0.0    /dev/sda        disk           26GB VBOX HARDDISK

```

Another way to display

the hardware information is to input the command **sudo lshw -class volume -short**. This command lists the volume/partition information.

```

└─(sarahben10@kali)-[/dev]
└─$ sudo lshw -class volume -short
H/W path          Device          Class          Description
-----
/0/100/d/0.0.0/1  /dev/sda1       volume         23GiB EXT4 volume
/0/100/d/0.0.0/2  /dev/sda2       volume         1359MiB Extended partition
/0/100/d/0.0.0/2/5 /dev/sda5       volume         1359MiB Linux swap volume

```

In part two of this lab, we focused on Linux hashing commands. Linux has many commands built into the system to hash files, drives, or strings. In order to hash a string, I used the following commands: **printf cs362 | sha1sum**, **printf cs362 | md5sum**, and **printf cs362 | sha512sum**.

The next command I

```
(sarahben10@kali)-[~]  
$ echo this is a text file > file1.txt  
  
(sarahben10@kali)-[~]  
$ md5sum file1.txt  
fda4e701258ba56f465e3636e60d36ec  file1.txt
```

If we wanted to hash all the files in a

```
(sarahben10@kali)-[~]  
$ md5sum Downloads/*  
fda4e701258ba56f465e3636e60d36ec Downloads/file1.txt
```

■ When using hash functions,

```
(sarahben10@kali)-[~]
$ printf cs362 | openssl dgst -sha3-256
SHA3-256(stdin)= e4ca8e0e958b39280f5ba86cd8864b194645c37ac1b89a778416a1bf23e4
ef0a

(sarahben10@kali)-[~]
$ openssl dgst -sha3-256 Downloads/*
SHA3-256(Downloads/file1.txt)= 5250c0bc0f9f9ebd8fa0789ffe426298188918c66488b22
84bd1eb376c96d2e84
```

In the next part of this lab, we concentrated on image acquisition using **dc3dd** and **dd** commands. The commands **dd**, **dc3dd**, and **dcfldd** all allow you to create a copy of the components of a hard disk. After installing dc3dd, I ran the dc3dd command to create a raw image of the /dev/sda2 drive. In that same command, I created a copy of the sda2 drive, hashed the content of that file, and logged the process into a file. I saved the image of the hard disk under the name **linux_image.dd**.

```

(kali@kali)-[~]
$ sudo dc3dd if=/dev/sda2 hash=sha1 log=linux_forensics.log of=linux_image.
dd

dc3dd 7.2.646 started at 2026-02-12 12:45:52 -0600
compiled options:
command line dc3dd if=/dev/sda2 hash=sha1 log=linux_forensics.log of=linux_im
age.dd
device size: 2 sectors (probed),          1,024 bytes
sector size: 512 bytes (probed)

      1024 bytes ( 1 K ) copied ( 100% ),    0 s, 9.2 K/s

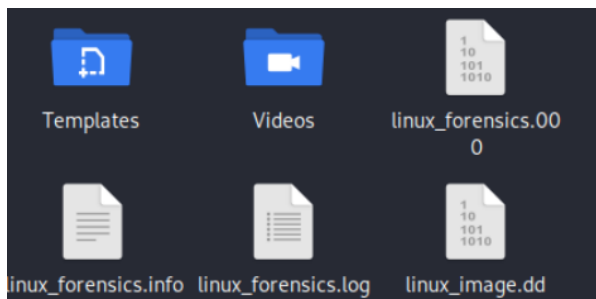
input results for device `/dev/sda2':
  2 sectors in
  0 bad sectors replaced by zeros
  0117fd74c20c3b8dac67136406e3c716ab536d49 (sha1)

output results for file `linux_image.dd':
  2 sectors out

dc3dd completed at 2026-02-12 12:45:52 -0600

```

If we needed to create an image of a disk that was very large, we could split the image into several different files. This is much easier to process and transfer when needed. To perform this task, we would input the following command: **sudo dc3dd if=/dev/sda2 hash=sha1 log=linux_forensics.info ofsz=550M ofs=linux_forensics.000**. The output in the terminal was the same as the screenshot above. When navigating to the file system, the files are shown as follows:



Because the sda2 drive isn't very large, only one file was created. To compute the hash functions of the split files, we use the command **cat linux_forensics.0* | sha1sum**.

```

(kali@kali)-[~]
$ cat linux_forensics.0* | sha1sum
0117fd74c20c3b8dac67136406e3c716ab536d49 -

```

This hash value is the same as the hash we created when using the dc3dd command. The dc3dd command can also be used to clear the contents of a particular drive. In order to do so, we will use the following commands to first overwrite the drive with zeros, then overwrite the drive with a pattern in hexadecimal, and lastly overwrite the drive with a text pattern.

```

(kali@kali)-[~]
$ sudo dc3dd wipe=/dev/sda2
[sudo] password for kali:

dc3dd 7.2.646 started at 2026-02-12 13:52:14 -0600
compiled options:
command line dc3dd wipe=/dev/sda2
device size: 2 sectors (probed),          1,024 bytes
sector size: 512 bytes (probed)

      1024 bytes ( 1 K ) copied ( 100% ),    0 s, 9.4 K/s

input results for pattern `00':
  2 sectors in

output results for device `/dev/sda2':
  2 sectors out

dc3dd completed at 2026-02-12 13:52:14 -0600

```

```

(kali@kali)-[~]
$ sudo dc3dd wipe=/dev/sda2 pat=ABCDEF

dc3dd 7.2.646 started at 2026-02-12 13:53:10 -0600
compiled options:
command line dc3dd wipe=/dev/sda2 pat=ABCDEF
device size: 2 sectors (probed),          1,024 bytes
sector size: 512 bytes (probed)

      1024 bytes ( 1 K ) copied ( 100% ),    0 s, 9.6 K/s

input results for pattern `ABCDEF':
  2 sectors in

output results for device `/dev/sda2':
  2 sectors out

dc3dd completed at 2026-02-12 13:53:10 -0600

```

```

(kali@kali)-[~]
$ sudo dc3dd wipe=/dev/sda2 tpat=happyholidays

dc3dd 7.2.646 started at 2026-02-12 13:53:52 -0600
compiled options:
command line dc3dd wipe=/dev/sda2 tpat=happyholidays
device size: 2 sectors (probed),          1,024 bytes
sector size: 512 bytes (probed)

      1024 bytes ( 1 K ) copied ( 100% ),    0 s, 8.7 K/s

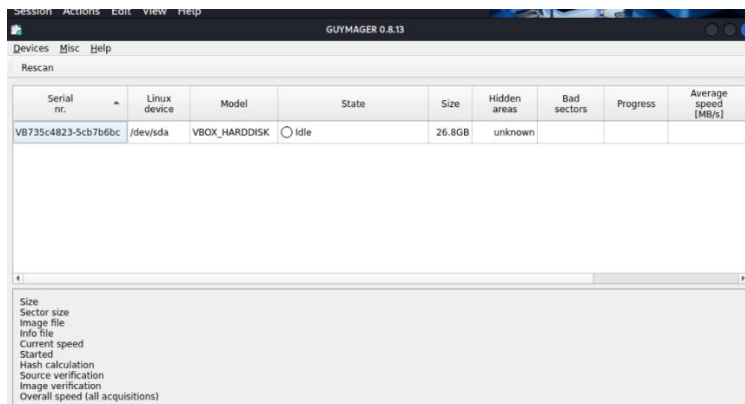
input results for pattern `happyholidays':
  2 sectors in

output results for device `/dev/sda2':
  2 sectors out

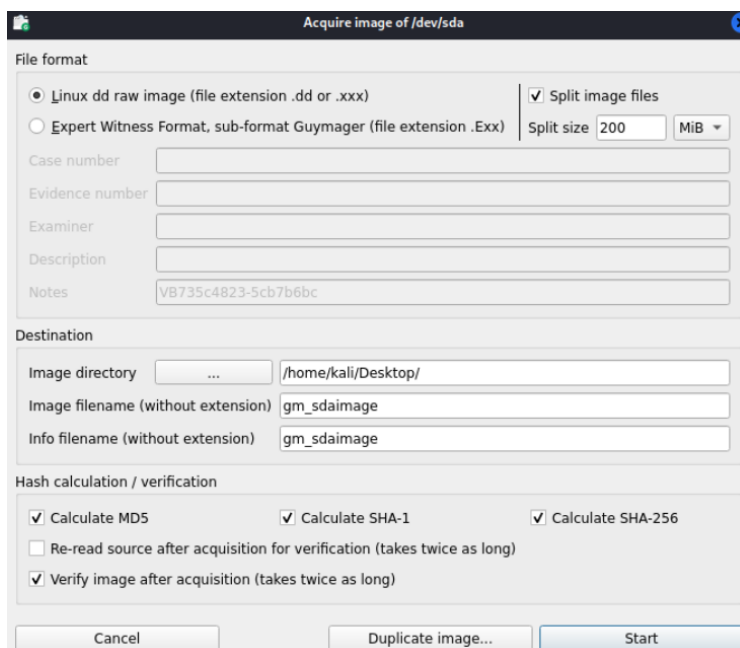
dc3dd completed at 2026-02-12 13:53:52 -0600

```

In the fourth part of this lab, we experiment with image acquisition using Guymager. To run guymager I used the command **sudo guymager**. Immediately after inputting the command, the guymager 0.8.13 window appears. The guymager lists all the detected drives along with their states, sizes, and other information found on the device being used. Next, I right-clicked on the /dev/sda drive and chose “acquire image” to select the format of the raw image file.



The window below appears once selecting “acquire image”. I chose the first option under file format, and chose the image directory and image filename.



Unfortunately, I did not have enough space on the virtual machine to complete this step, and an error occurred starting: “aborted – image write error”.

Serial nr.	Linux device	Model	State	Size	Hidden areas	Bad sectors	Progress	Average speed [MB/s]
VB735c4823-5cb7b6bc	/dev/sda	VBOX_HARDDISK	Aborted - Image write error	26.8GB	unknown	0	17%	24.15

The second-to-last part of this lab focuses on retrieving the Master Boot Record (MBR) using the dd command. In order to retrieve the first cluster of the drive, I used the command: **sudo dd if=/dev/sda bs=512 of=mbr.image count=1.**

```
(kali@kali)-[~]
$ sudo dd if=/dev/sda bs=512 of=mbr.image count=1
[sudo] password for kali:
1+0 records in
1+0 records out
512 bytes copied, 0.0039369 s, 130 kB/s
```

The output lists the records in and out as well as the bytes copied. The last step in this part instructs us to open the file mbr.image in HxD

to examine its contents. Since the HxD software only works through Windows, we input the following command into the terminal to retrieve the same information that would be obtained from HxD: `xxd -g 1 mbr.image`.

```
(kali㉿kali)-[~]
$ xxd -g 1 mbr.image
00000000: eb 63 90 10 8e d0 bc 00 b0 b8 00 00 8e d8 8e c0 .C.....
00000010: fb be 00 7c bf 00 06 b9 00 02 f3 a4 ea 21 06 00 ..|.!.
00000020: 00 be be 07 38 04 75 0b 83 c6 10 81 fe fe 07 75 ...8.u...u
00000030: f3 eb 16 b4 02 b0 01 bb 00 7c b2 80 8a 74 01 8b .....|...t..
00000040: 4c 02 cd 13 ea 00 7c 00 00 eb fe 00 00 00 00 00 L.....|..
00000050: 00 00 00 00 00 00 00 00 00 00 80 01 00 00 00 .....
00000060: 00 00 00 ff fa 90 90 f6 c2 80 74 05 f6 c2 70 .....t...p
00000070: 74 02 b2 80 ea 79 7c 00 00 31 c0 8e d8 8e d0 bc t...y|..1...
00000080: 00 20 fb a0 64 7c 3c ff 74 02 88 c2 52 be 80 7d . ..d|<.t...R..}
00000090: e8 17 01 be 05 7c b4 41 bb aa 55 cd 13 5a 52 72 .....|.A..U..ZRR
000000a0: 3d 81 fb 55 aa 75 37 83 e1 01 74 32 31 c0 89 44 =..U.u7...t21..D
000000b0: 04 40 88 44 ff 89 44 02 c7 04 10 00 66 8b 1e 5c .@.D..D...f..\
000000c0: 7c 66 89 5c 08 66 8b 1e 60 7c 66 89 5c 0c c7 44 |f..\f..`|f..\D
000000d0: 06 00 70 b4 42 cd 13 72 05 bb 00 70 eb 76 b4 08 ..p.B..r..p.v..
000000e0: cd 13 73 0d 5a 84 d2 0f 83 d8 00 be 8b 7d e9 82 ..s.Z.....}..
000000f0: 00 66 0f b6 c6 88 64 ff 40 66 89 44 04 0f b6 d1 .f....d.@f.D...
00000100: c1 e2 02 88 e8 88 f4 40 89 44 08 0f b6 c2 c0 e8 .....@.D.....
00000110: 02 66 89 04 66 a1 60 7c 66 09 c0 75 4e 66 a1 5c .f..f..|f..uNf.\
00000120: 7c 66 31 d2 66 f7 34 88 d1 31 d2 66 f7 74 04 3b |f1.f.4..1.f.t.;
00000130: 44 08 7d 37 fe c1 88 c5 30 c0 c1 e8 02 08 c1 88 D.}7...0.....
00000140: d0 5a 88 c6 bb 00 70 8e c3 31 db b8 01 02 cd 13 .Z....p..1.....
00000150: 72 1e 8c c3 60 1e b9 00 01 8e db 31 f6 bf 00 80 r...`...1...
00000160: 8e c6 fc f3 a5 1f 61 ff 26 5a 7c be 86 7d eb 03 .....a.5Z|..}..
00000170: be 95 7d e8 34 00 be 9a 7d e8 2e 00 cd 18 eb fe ..}.4..}.....
00000180: 47 52 55 42 20 00 47 65 6f 6d 00 48 61 72 64 20 GRUB .Geom.Hard
00000190: 44 69 73 6b 00 52 65 61 64 00 20 45 72 72 6f 72 Disk.Read. Error
000001a0: 0d 0a 00 bb 01 00 b4 0e cd 10 ac 3c 00 75 f4 c3 ..<..u...
000001b0: 00 00 00 00 00 00 00 00 90 f0 b0 08 00 00 80 04 .....
000001c0: 01 04 83 fe c2 ff 00 08 00 00 70 f5 02 00 fe .....p...
000001d0: c2 ff 0f fe c2 ff fe 7f f5 02 02 78 2a 00 00 00 .....X*...
000001e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa .....U..
```

The first column lists the

indices, the second column lists the hexadecimal, and the third column displays the text of the file. The signature of the Master Boot Record is **55 aa**.

In the last part of this lab, we experimented with Windows disk investigation using PowerShell cmdlets. To begin, I searched for PowerShell gallery using the keyword “forensic”. I ran into a few errors because the PowerForensics modules weren’t installed in my Windows virtual machine. I had to use ChatGPT to troubleshoot this because I wasn’t finding an answer through Google. After some time, I successfully installed the module and was able to insert the first command.

```
PS C:\Windows\system32> Find-Module -Name *forensic*
WARNING: '.', hexadecimal value 0x00, is an invalid character. Line 1, position 1.

Version      Name                Repository      Description
-----
1.1.1        PowerForensics      PSGallery       A Digital Forensics framework for
1.1.1        PowerForensicsv2    PSGallery       A Digital Forensics framework for
1.1.1        PowerForensicsPortable PSGallery       A Digital Forensics framework for
1.0.0.0      Forensics            PSGallery       The module can be used for perfor
```

Once hitting

enter, a list of modules and scripts appeared in a list. The next command we used is **Get-ChildItem**, and this displays a list of installed modules.

```
PS C:\Windows\system32> Get-ChildItem -Path 'C:\Program Files\WindowsPowerShell\Modules'

Directory: C:\Program Files\WindowsPowerShell\Modules

Mode                LastWriteTime         Length Name
----                -
d-----         12/7/2019   3:31 AM                Microsoft.PowerShell.Operation.Validation
d-----         12/7/2019   3:31 AM                PackageManagement
d-----         12/7/2019   3:31 AM                Pester
d-----         12/7/2019   3:31 AM                PowerShellGet
d-----         12/7/2019   3:31 AM                PSReadline
```

When using -Force with

the Get-ChildItem command, the same output appeared as the screenshot above. And when inserting -Recurse with the same command, an extremely long list appeared. I added a screenshot below of only part of that list since it was so lengthy.

```
PS C:\Windows\system32> Get-ChildItem -Recurse 'C:\Program Files\WindowsPowerShell\Modules'

Directory: C:\Program Files\WindowsPowerShell\Modules

Mode                LastWriteTime         Length Name
----                -
d-----         12/7/2019   3:31 AM                Microsoft.PowerShell.Operation.Validation
d-----         12/7/2019   3:31 AM                PackageManagement
d-----         12/7/2019   3:31 AM                Pester
d-----         12/7/2019   3:31 AM                PowerShellGet
d-----         12/7/2019   3:31 AM                PSReadline

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation

Mode                LastWriteTime         Length Name
----                -
d-----         12/7/2019   3:31 AM                1.0.1

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation\1.0.1

Mode                LastWriteTime         Length Name
----                -
d-----         12/7/2019   3:31 AM                Diagnostics
d-----         12/7/2019   3:14 AM                Test
-a-----         12/7/2019   3:10 AM        3146 Microsoft.PowerShell.Operation.Validation.Format.ps1xml
-a-----         12/7/2019   3:10 AM        1556 Microsoft.PowerShell.Operation.Validation.psd1
-a-----         12/7/2019   3:10 AM        12310 Microsoft.PowerShell.Operation.Validation.psm1
-a-----         12/7/2019   3:10 AM          347 operationValidationResources.psd1

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation\1.0.1\Diagnostics
```

```
Mode                LastWriteTime         Length Name
----                -
d-----         12/7/2019   3:14 AM                Comprehensive
d-----         12/7/2019   3:14 AM                Simple

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation\1.0.1\Diagnostics\Comprehensive

Mode                LastWriteTime         Length Name
----                -
-a-----         12/7/2019   3:10 AM        759 Comprehensive.Tests.ps1

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation\1.0.1\Diagnostics\Simple

Mode                LastWriteTime         Length Name
----                -
-a-----         12/7/2019   3:10 AM          384 Simple.Tests.ps1

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation\1.0.1\Test

Mode                LastWriteTime         Length Name
----                -
d-----         12/7/2019   3:31 AM                Modules
-a-----         12/7/2019   3:10 AM        5046 Microsoft.PowerShell.Operation.Validation.Tests.ps1

Directory: C:\Program Files\WindowsPowerShell\Modules\Microsoft.PowerShell.Operation.Validation\1.0.1\Test\Modules
```

The last few commands deal with the PowerForensics module. I inserted the following command to display a list of the module's contained cmdlets.

```
PS C:\Windows\system32> Get-Command -Module PowerForensics
```

CommandType	Name	Version	Source
Cmdlet	ConvertFrom-BinaryData	1.1.1	PowerForensics
Cmdlet	ConvertTo-ForensicTimeline	1.1.1	PowerForensics
Cmdlet	Copy-ForensicFile	1.1.1	PowerForensics
Cmdlet	Get-ForensicAlternateDataStream	1.1.1	PowerForensics
Cmdlet	Get-ForensicAmcache	1.1.1	PowerForensics
Cmdlet	Get-ForensicAttrDef	1.1.1	PowerForensics
Cmdlet	Get-ForensicBitmap	1.1.1	PowerForensics
Cmdlet	Get-ForensicBootSector	1.1.1	PowerForensics
Cmdlet	Get-ForensicChildItem	1.1.1	PowerForensics
Cmdlet	Get-ForensicContent	1.1.1	PowerForensics
Cmdlet	Get-ForensicEventLog	1.1.1	PowerForensics
Cmdlet	Get-ForensicExplorerTypedPath	1.1.1	PowerForensics
Cmdlet	Get-ForensicFileRecord	1.1.1	PowerForensics
Cmdlet	Get-ForensicFileRecordIndex	1.1.1	PowerForensics
Cmdlet	Get-ForensicFileStack	1.1.1	PowerForensics
Cmdlet	Get-ForensicGuidPartitionTable	1.1.1	PowerForensics
Cmdlet	Get-ForensicMasterBootRecord	1.1.1	PowerForensics
Cmdlet	Get-ForensicMftSlack	1.1.1	PowerForensics
Cmdlet	Get-ForensicNetworkList	1.1.1	PowerForensics
Cmdlet	Get-ForensicOfficeFileMru	1.1.1	PowerForensics
Cmdlet	Get-ForensicOfficeOutlookCatalog	1.1.1	PowerForensics
Cmdlet	Get-ForensicOfficePlaceMru	1.1.1	PowerForensics
Cmdlet	Get-ForensicOfficeTrustRecord	1.1.1	PowerForensics
Cmdlet	Get-ForensicPartitionTable	1.1.1	PowerForensics
Cmdlet	Get-ForensicPrefetch	1.1.1	PowerForensics
Cmdlet	Get-ForensicRecentFileCache	1.1.1	PowerForensics
Cmdlet	Get-ForensicRegistryKey	1.1.1	PowerForensics
Cmdlet	Get-ForensicRegistryValue	1.1.1	PowerForensics
Cmdlet	Get-ForensicRunKey	1.1.1	PowerForensics
Cmdlet	Get-ForensicRunMru	1.1.1	PowerForensics
Cmdlet	Get-ForensicScheduledJob	1.1.1	PowerForensics
Cmdlet	Get-ForensicShellLink	1.1.1	PowerForensics

This list was also very long, so I only included a small snippet of it. The next command we experimented with was **Get-ForensicVolumeBootRecord**. This command allows the user to retrieve the master boot record of a volume.

```
PS C:\Windows\system32> Get-ForensicVolumeBootRecord -VolumeName \\.\C: -AsBytes | Format-Hex
```

```

00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000 EB 52 90 4E 54 46 53 20 20 20 20 00 02 08 00 00  eRENTFS .....
00000010 00 00 00 00 00 F8 00 00 3F 00 FF 00 00 08 00 00  .....o..?.....
00000020 00 00 00 00 80 00 80 00 FF EF 3F 06 00 00 00 00  .....@.@..i?....
00000030 00 00 0C 00 00 00 00 00 02 00 00 00 00 00 00 00  .....
00000040 F6 00 00 00 01 00 00 86 77 12 46 94 12 46 D4 00  ö.....w.FB.FÖ
00000050 00 00 00 00 FA 33 C0 8E D0 8C 00 7C FB 68 C0 07  ...ú3Ä@X.|ÜhÄ.
00000060 1F 1E 68 66 00 C8 88 16 0E 00 66 81 3E 03 00 4E  ..hf.E@...f@>..N
00000070 54 46 53 75 15 B4 41 B8 AA 55 CD 13 72 0C 81 FB  TFSu."A"UÍ,r.ÜÜ
00000080 55 AA 75 06 F7 C1 01 00 75 03 E9 DD 00 1E 83 EC  U#u.±Ä..u.éY..@1
00000090 18 68 1A 00 B4 48 8A 16 0E 00 88 F4 16 1F CD 13  .h."H@...@ö..f.
000000A0 9F 83 C4 18 9E 58 1F 72 E1 3B 06 0B 00 75 D8 A3  @BÄ.®X.rá;...uÜE
000000B0 0F 00 C1 2E 0F 00 04 1E 5A 33 D8 B9 00 20 28 C8  ..Ä.....Z3Ü¹. +È
000000C0 66 FF 06 11 00 03 16 0F 00 8E C2 FF 06 16 00 E8  f.....@Ä...è
000000D0 48 00 28 C8 77 EF B8 00 B8 CD 1A 66 23 C0 75 2D  K.+ÈwI..uÍ.f#Au-
000000E0 66 81 FB 54 43 50 41 75 24 81 F9 02 01 72 1E 16  f@UTCPAu$@U..r...
000000F0 68 07 B8 16 68 52 11 16 68 09 00 66 53 66 53 66  h.».hR..h..fSfSf
00000100 55 16 16 16 68 B8 01 66 61 0E 07 CD 1A 33 C0 BF  U...h..fa..Í.3A¿
00000110 0A 13 B9 F6 0C FC F3 AA E9 FE 01 90 90 66 60 1E  ..¹@.ü@ép.®®F¹.
00000120 06 66 A1 11 00 66 03 06 1C 00 1E 66 68 00 00 00  .fj..f.....fh...
00000130 00 66 50 06 53 68 01 00 68 10 00 B4 42 8A 16 0E  .fP.Sh..h..¹B@...
00000140 00 16 1F 88 F4 CD 13 66 59 5B 5A 66 59 66 59 1F  ...@ÖÍ.fY[ZfYfY.
00000150 0F 82 16 00 66 FF 06 11 00 03 16 0F 00 8E C2 FF  .@..f.....@Ä.
00000160 0E 16 00 75 BC 07 1F 66 61 C3 A1 F6 01 E8 09 00  ...uX..faÄj@è..
00000170 A1 FA 01 E8 03 00 F4 EB FD 8B F0 AC 3C 00 74 09  jü.è..öëY@ö-<.t.
00000180 B4 0E B8 07 00 CD 10 EB F2 C3 0D 0A 41 20 64 69  ".»..Í.èBÄ..A di
00000190 73 68 20 72 65 61 64 20 65 72 72 6F 72 20 6F 63  sk read error oc
000001A0 63 75 72 72 65 64 00 00 0A 42 4F 4F 54 4D 47 52  currend...BOOTMGR
000001B0 20 69 73 20 63 6F 6D 70 72 65 73 73 65 64 00 00  .is compressed..
000001C0 0A 50 72 65 73 73 20 43 74 72 6C 2B 41 6C 74 2B  .Press Ctrl+Alt+
000001D0 44 65 6C 20 74 6F 20 72 65 73 74 61 72 74 00 0A  Del to restart..
000001E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
000001F0 00 00 00 00 00 8A 01 A7 01 BF 01 00 00 55 AA  ....@.g.¿...U²

```

The last command of this lab deals with the command **Get-ForensicFileRecord**. This command is used to display the file records from the master file table (MFT). Since I saved the cat and dog .png files to my actual computer for the last lab, I was unable to perform this last part within the Windows virtual machine.

Part 3: References

1. Module 4: Disks, File Systems, and Media Devices lecture recording
2. Lab 4 Disk and File Systems